

Relazione sintetica - Progetto Connections

1. Obiettivo e architettura generale

Il progetto realizza il gioco Connections con architettura client-server centralizzata. Il server mantiene l'unica partita globale attiva, lo stato degli utenti, lo storico dei risultati e la classifica; i client offrono una CLI e comunicano tramite una connessione TCP persistente con messaggi JSON delimitati da newline. Le notifiche asincrone di cambio/fine partita sono inviate via UDP, ma lo stato autorevole resta sempre interrogabile via TCP.

La scelta architetturale principale è la separazione dei piani: I/O non bloccante, logica applicativa e servizi periodici non sono mescolati. Ogni thread ha una responsabilità limitata e ogni struttura condivisa ha un proprietario o una primitiva di sincronizzazione esplicita.

Area	Responsabilità	Scelte progettuali valorizzate
shared.protocol	Contratto JSON comune: Operation, StatusCode, Request, Response.	Evita stringhe sparse, centralizza i nomi dei campi e conserva requestId per correlare risposte concorrenti.
client	CLI, rendering testuale, ConnectionManager NIO, UdpListener.	API utente quasi sincrona sopra rete asincrona: Future, selector e requestId nascondono la complessità.
server	Networking, sicurezza, gioco, sessioni, leaderboard, persistenza, servizi temporizzati.	Reactor + WorkerPool + servizi schedulati; dati primari persistenti e viste derivate ricostruibili.

2. Scelte progettuali principali

Protocollo JSON su TCP persistente: Ogni comando viaggia come JsonObject UTF-8 terminato da \n. Il newline è una scelta semplice dato che TCP è uno stream: MessageAccumulator ricostruisce messaggi completi anche se una read contiene frammenti o più JSON.

requestId è la chiave di indicizzazione: Client e server non assumono che le risposte arrivino nello stesso ordine delle richieste. ConnectionManager associa requestId -> CompletableFuture e completa la Future corretta quando la response torna.

TCP per comandi, UDP per eventi: TCP resta il canale affidabile per register, login, play, stats e ranking. UDP è usato solo per notifiche best-effort recuperabili tramite info/stats TCP, quindi la perdita di un datagramma non corrompe lo stato.

Identità stabile con playerId: Lo username è modificabile; lo storico usa playerId. Questa scelta protegge risultati, leaderboard e statistiche da rename e reconnect.

Persistenza minima e ricostruzione: Vengono salvati utenti e PlayerGameResult; leaderboard e statistiche sono viste derivate. Al riavvio PersistenceService ricarica il dato primario e ricostruisce lo stato derivato.

Gestione file giochi in streaming: GameLoader/StorageManager evitano di trattare il JSON come un blocco unico quando non serve. Questa scelta rispetta l'idea della consegna: il file delle partite va gestito come potenzialmente grande.

Sicurezza applicativa stratificata (non richiesta ma è stato divertente progettarla): Limite dei frame, parsing controllato, RequestGuard per rate limit/ban, RequestValidator prima degli handler, password con salt+hash. Limite dichiarato: il canale non è cifrato, non volevo implementare TLS.

3. Schema generale dei thread attivati

Lato	Thread	Attesa / sblocco	Responsabilità
Server	main	join() / shutdown	Wiring dei componenti, avvio runtime, installazione shutdown ordinato.
Server	acceptor	Selector.select() su OP_ACCEPT	Accetta socket, crea ChannelContext e assegna round-robin agli IODispatcher.
Server	io-dispatcher-i	Selector.select(), wakeup da add/requestWrite	Gestisce OP_WRITE e OP_READ, si occupa del framing dei messaggi e delega la logica cpu bound ai worker
Server	worker-n	ThreadPoolExecutor queue	Esegue RequestRouter e handler; produce JsonObject e lo accoda, non scrive direttamente sul socket.
Server	game-timer	ScheduledExecutorService	Scandisce la partita globale, finalizza timeout e invia notifiche UDP.
Server	persistence	scheduleAtFixedRate	Load iniziale, salvataggi periodici, save finale, rebuild di statistiche e classifica.
Server	session-monitor	scheduleAtFixedRate	Riconnessioni scadute, inattività, cleanup di RequestGuard.
Client	main / CLI	Scanner o Future.get(timeout)	Legge comandi, mantiene loggedIn/currentUsername, invoca ask e usa un render per l'output.
Client	client-selector	Selector.select()	Scrittura fisica TCP, lettura response, completion delle Future pendenti.
Client	udp-listener	DatagramChannel.receive()	Riceve notifiche, stampa in modo

			ordinato e può richiedere automaticamente gameInfo.
--	--	--	---

4. Divisione per directory

Cartella	Classi principali	Scopo
src/shared/protocol	Operation, Request, Response, StatusCode	Definisce il protocollo una volta sola; riduce errori fra client e server e mantiene i codici applicativi documentabili.
src/client	ClientMain, ClientConfig	Bootstrap e configurazione separati dall'interazione utente: porta/timeout/buffer stanno nei file config.
src/client/cli	CommandLoop, CliRenderer, AnsiCodes	CLI ordinata: outputLock evita interleaving fra prompt, comandi e UDP; renderer separa view da logica.
src/client/net	ConnectionManager, UdpListener	TCP NIO asincrono con pendingRequests/pendingWrites; UDP in thread dedicato per eventi asincroni.
src/server	ServerMain, ServerConfig	In root: Main carica config, costruisce dipendenze, avvia thread, componenti e shutdown.
src/server/net	Acceptor, IODispatcher, ChannelContext, MessageAccumulator, UdpNotifier	Reactor NIO completo: accept separato, dispatcher multipli round robin, contesto per connessione, framing, write queue e notifiche.
src/server/core	RequestRouter, WorkerPool	Ponte fra I/O e tutto il resto: routing per Operation e pool limitato con backpressure strategica.
src/server/handler	9 handler applicativi	Ogni caso d'uso è isolato
src/server/game	GameManager, GameLoader, GameState, GameResultRepository, PlayerGameResult	Il gioco: partita globale, storico playerId/gameId, statistiche ricostruibili.
src/server/session	SessionRegistry, PlayerSession, PlayerGameState, SessionState	Identità, autenticazione, reconnect e stato per-player separati dal canale TCP.
src/server/leaderboard	Leaderboard	Vista ordinata concorrente basata su ScoreKey, ricostruibile dai risultati.
src/server/security	PasswordHasher, RequestGuard, RequestValidator	Sicurezza difensiva introdotta prima per gioco poi seriamente: hash password, rate limit, validazione strutturale per protezione da attacchi Ddos.
src/server/service	GameTimerService, PersistenceService, SessionMonitorService	Tempo, persistenza e manutenzione sono servizi dedicati, non codice dentro gli handler.
src/server/storage	ByteBufferPool, StorageManager	Buffer diretti per TCP (giusto per usare direct buffer ottimizzati) e JSON streaming/atomico per storage: due problemi risolti con strumenti diversi.

5. Strutture dati e primitive di sincronizzazione

La sincronizzazione non è concentrata in un unico lock globale: il progetto usa strutture concorrenti per dati indipendenti, lock locali per operazione inevitabilmente composte e nient'altro quando un oggetto è posseduto da un solo thread.

Precisazione sul "thread sbagliato": in Java NIO un Selector va trattato come oggetto posseduto dal thread che lo esegue. Per questo Acceptor non registra direttamente il SocketChannel sul Selector di un IODispatcher: crea il ChannelContext, lo inserisce nella pendingRegistrations del dispatcher scelto e chiama selector.wakeup(). Quando il dispatcher si risveglia, drena la coda e chiama channel.register(selector, OP_READ, ctx) nel thread corretto. Lo stesso vale per le risposte: i worker accodano ByteBuffer nel ChannelContext e segnalano pendingWriteKeys; solo lo IODispatcher abilita OP_WRITE e scrive fisicamente sulla socket.

Componente	Strutture / primitive	Motivazione
IODispatcher	ConcurrentLinkedQueue pendingRegistrations/pendingWriteKeys; SelectionKey; ByteBufferPool	Gestione cross-thread acceptor/worker -> dispatcher: Acceptor e worker non modificano direttamente il Selector di un altro thread. Mettono invece lavoro in code concorrenti e chiamano wakeup(); il thread io-dispatcher proprietario registra i SocketChannel e abilita OP_WRITE sul proprio Selector.
ChannelContext	ConcurrentLinkedQueue<ByteBuffer> writeQueue; session volatile; MessageAccumulator confinato	Worker producono output, dispatcher consuma; il framer non richiede lock perché ha un solo proprietario.
WorkerPool	ThreadPoolExecutor + ArrayBlockingQueue + CallerRunsPolicy	Coda limitata e backpressure ottimale, alle brutte esegue il thread dispatcher.
SessionRegistry	ConcurrentHashMap byId/IdByName; AtomicLong; metodi register/rename/snapshot synchronized	Indici multipli coerenti e generazione playerId stabile.
PlayerSession	AtomicReference<SessionState>; campi volatile; stats synchronized; gameLock	Transizioni atomiche, visibilità dei campi semplici e snapshot coerente delle statistiche.
PlayerGameState	metodi synchronized	Due proposal dello stesso player sono serializzate; player diversi non si bloccano a vicenda.
GameManager	ReentrantReadWriteLock; currentGame volatile; AtomicInteger	Molte read/proposal convivono ma advance/expire prendono writeLock e proteggono la sostituzione della partita.
GameResultRepository	ReadWriteLock + mappe byKey/byPlayer/byGame/score	addResult è idempotente su playerId+gameId; le query storiche usano readLock.

Leaderboard	ConcurrentSkipListMap + ConcurrentHashMap	Top/rank efficienti, indicizzati e ordinati seppur rimanendo soltanto una vista non uno storico
RequestGuard	ConcurrentHashMap + AtomicInteger	Rate limit e ban per IP senza bloccare tutto il server.
Client ConnectionManager	ConcurrentHashMap pendingRequests; ConcurrentLinkedQueue pendingWrites; AtomicLong requestId	Separazione fra mappa logica della request e lista fisica dei bytes; gestione grazie a request id per risposte fuori ordine.
Client CLI	outputLock / metodi synchronized del renderer	Terminale coerente anche con notifiche UDP mentre il main attende una Future.

6. Flussi runtime significativi

Request TCP completa: CommandLoop costruisce Request -> ConnectionManager assegna un requestId atomico, mette Future in pendingRequests e ByteBuffer in pendingWrites -> client-selector scrive -> IODispatcher del server legge, accumula e valida -> WorkerPool esegue router/handler -> ChannelContext.write accoda la response -> IODispatcher scrive -> client-selector legge e completa la Future corretta.

Submit proposal: SubmitProposalHandler invoca GameManager.check sotto readLock; PlayerGameState aggiorna gruppi/errori/score in synchronized; se il player vince, perde o scade il timer, GameResultRepository consolida una sola volta il risultato e Leaderboard aggiorna la vista.

Scadenza partita: GameTimerService prende il controllo temporale: expireCurrentGame finalizza i player non conclusi, poi UdpNotifier invia game_ended; dopo l'intervallo parte advanceGame che mantiene thread-safeness grazie al monitor GameLock e i logged-in entrano automaticamente nella nuova partita.

Persistenza e riavvio: PersistenceService carica utenti e risultati; StorageManager tollera file mancanti/vuoti/non validi come stato iniziale; al boot ricostruisce leaderboard/stats e GameManager riesce anche a ripartire dal primo gameId successivo allo storico.

7. Compilazione, esecuzione e verifica

Prerequisiti: JDK con javac/java; libreria esterna già inclusa in MyApp/lib/gson-2.10.1.jar; permessi di esecuzione dalla directory MyApp.

I file di configurazione sono config/server_config.json e config/client_config.json;

```
cd MyApp
./run-server.sh
./run-client.sh
```

```
# Compilazione manuale equivalente:
find src -type f -name "*.java" > .all_sources.tmp
javac -cp lib/gson-2.10.1.jar -d out @.all_sources.tmp
java -cp out:lib/gson-2.10.1.jar server.ServerMain
java -cp out:lib/gson-2.10.1.jar client.ClientMain
```

Comandi client: register <user> <pass>, login <user> <pass>, logout, update user | pass | both, play | p <w1 | "w1"> <w...> <w4>, info [current | gameId], stats [current | gameId], top [k], rank <player>, me, help, quit